



Department of Artificial Intelligence and Machine Learning

Experiment No.: 1

Experiment on Amortized Analysis.

Aim

To perform Amortized Analysis using Aggregate, Accounting and Potential Methods.

Objectives

- To understand the concept of amortized analysis.
- To analyze the average cost of operations over a sequence of operations.
- To study Aggregate Method.
- To study Accounting Method.
- To study Potential Method.
- To compare all three amortized analysis techniques.

Theory

Amortized analysis is a technique used in algorithm analysis to determine the average time complexity per operation over a sequence of operations rather than considering the worst-case complexity of a single operation. It guarantees that although some operations may be expensive, the average cost remains low over time.

Unlike average-case analysis, amortized analysis does not depend on probability. Instead, it provides a deterministic bound on the total cost of performing a sequence of operations.

There are three common methods of amortized analysis:

1. Aggregate Method

The Aggregate Method analyzes the total cost of a sequence of operations and divides it by the total number of operations.

Suppose an algorithm performs n operations with a total cost $T(n)$.

The amortized cost is



$$\text{Amortized Cost} = \frac{T(n)}{n}$$

For example, if inserting n elements into a dynamic array requires a total of $3n$ operations,

$$\text{Amortized Cost} = \frac{3n}{n} = 3 = O(1)$$

Thus, although resizing occasionally takes linear time, each insertion has constant amortized complexity.

2. Accounting Method

The Accounting Method assigns an artificial cost (or charge) to each operation.

- Cheap operations are charged extra.
- The extra amount is stored as credit.
- Expensive operations use the accumulated credit.

The total credit is always non-negative.

For example,

Actual insertion cost

$$= 1$$

Charged cost

$$= 3$$

Extra credit stored

$$= 2$$

When resizing occurs, the stored credits pay for copying the elements.

This method proves that every insertion has

$$O(1)$$

amortized complexity.

3. Potential Method

The Potential Method assigns a potential value to the data structure.

Potential represents stored work that can pay for future expensive operations.

The amortized cost is calculated as



$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

where

- c_i = Actual cost
- $\Phi(D_i)$ = Potential after operation
- $\Phi(D_{i-1})$ = Potential before operation

A commonly used potential function for a dynamic array is

$$\Phi = 2 \times \text{Number of Elements} - \text{Capacity}$$

This ensures that the amortized cost remains constant even when resizing occurs.

Algorithm

Step 1: Start.

Step 2: Initialize a dynamic array.

Step 3: Insert elements one by one.

Step 4: Double the array size whenever it becomes full.

Step 5: Calculate:

- Actual Cost
- Aggregate Cost
- Accounting Cost
- Potential Cost

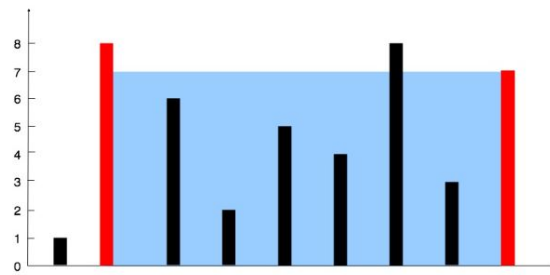
Step 6: Display the amortized costs.

Step 7: Stop.

Problem Statement

1. You are given an integer array height of length n. There are n vertical lines drawn such that the two endpoints of the ith line are (i, 0) and (i, height[i]). Find two lines that together with the x-axis form a container, such that the container contains the most water. Return the maximum amount of water a container can store. Notice that you may not slant the container.

Example 1:



Input: height = [1,8,6,2,5,4,8,3,7]

Output: 49

Explanation: The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case, the max area of water (blue section) the container can contain is 49.

Figure 1: Container With Most Water Illustration

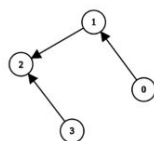
2. You are maintaining a project that has n methods numbered from 0 to $n - 1$. You are given two integers n and k , and a 2D integer array `invocations`, where `invocations[i] = [ai, bi]` indicates that method a_i invokes method b_i . There is a known bug in method k . Method k , along with any method invoked by it, either directly or indirectly, are considered suspicious and we aim to remove them. A group of methods can only be removed if no method outside the group invokes any methods within it. Return an array containing all the remaining methods after removing all the suspicious methods. You may return the answer in any order. If it is not possible to remove all the suspicious methods, none should be removed.

Example 1:

Input: $n = 4$, $k = 1$, `invocations = [[1,2], [0,1], [3,2]]`

Output: [0,1,2,3]

Explanation:



Method 2 and method 1 are suspicious, but they are directly invoked by methods 3 and 0, which are not suspicious. We return all elements without removing anything.

Figure 2: Project Illustration

Viva Questions

1. What is amortized analysis?
2. Why is amortized analysis better than worst-case analysis?
3. What is the Aggregate Method?
4. Explain the Accounting Method.



5. What is a potential function?
6. Give examples where amortized analysis is used.
7. Differentiate between amortized and average-case analysis.

References

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th Edition, MIT Press, 2022. (Chapter 17: Amortized Analysis)
2. Mark Allen Weiss, *Data Structures and Algorithm Analysis in C++*, 4th Edition, Pearson Education, 2014.
3. Ellis Horowitz, Sartaj Sahni, and Susan Anderson-Freed, *Fundamentals of Data Structures in C++*, 2nd Edition, Universities Press, 2008.
4. Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman, *Data Structures and Algorithms*, Addison-Wesley, 1983.
5. MIT OpenCourseWare, "Introduction to Algorithms (Amortized Analysis)", Available at: <https://ocw.mit.edu/courses/6-006-introduction-to-algorithms/>